INTELLIGENT LEARNING ANALYTICS

& Agentic AI Study Coach

Milestone 2 — Agentic AI Study Coach

End-Semester Submission | Gen AI Course

Team Members

Sathvik Koriginja — 2401010231

Anushka Tyagi — 2401010090

Apoorva Choudhary — 2401010092

Live Application: https://predictive-learning-analytics-ml.streamlit.app/

GitHub Repository: https://github.com/sathvik89/Predictive-Learning-Analytics_ML

Video Demo: https://drive.google.com/file/d/17Jmu-UJTZktNFDcbaib-NfwwbhWNz3GP/view?usp=sharing

1. Project Overview

Milestone 1 built a machine learning pipeline that could predict a student's exam score, classify them as Pass or Fail, and group them into one of three learner categories — At-Risk, Average, or High-Performer. That was all done through a static dashboard where you filled in sliders and got back a result.

Milestone 2 takes the same ML models and wraps them inside a conversational AI agent. Instead of a form, the student just types naturally. The agent reads what they said, figures out what they need (analysis, a study plan, an explanation, a quiz), and responds accordingly. It also remembers what was said earlier in the conversation and saves session history to a database so previous chats can be resumed.

The agent is built using LangGraph, which lets us define a graph of specialised nodes, each responsible for one job. A central master node decides which node to visit based on what the user is asking. This is what makes the system genuinely agentic rather than just a chatbot with a smart prompt.

1.1 What Milestone 1 Delivered

Three models were trained on 30,640 student records from the Students Exam Scores Extended dataset (Kaggle):

Model	Task	Result
Linear Regression	Predict ExamScore (0–100)	$R^2 = 0.9397$, MAE = 3.04 marks
Logistic Regression	Classify Pass or Fail	Accuracy = 91.76%, F1 = 0.9176
K-Means Clustering	Segment into learner categories	Silhouette = 0.2112 at k = 3

These six trained model files (three models, three scalers) are carried directly into Milestone 2. The Analyser node calls them every time a student shares their scores.

1.2 What Milestone 2 Adds

- A conversational interface where the student types naturally instead of using sliders
- A multi-node LangGraph agent that routes each message to the right specialist
- A RAG system that retrieves verified study content before generating advice
- A web search fallback when the local knowledge base does not cover the topic
- An interactive 5-question quiz graded automatically
- A personalised 7-day study plan built from ML results and retrieved knowledge
- Academic guardrails that keep the system focused and block misuse
- Full session memory within a conversation and persistent chat history across sessions via PostgreSQL

2. System Architecture

The system has five layers that work together. The student types into the Streamlit interface. The message goes into the LangGraph agent. The agent runs one or more specialist nodes. Those nodes call ML tools, search the knowledge base, or generate a plan. The response comes back to the chat. Everything said is saved to PostgreSQL.

2.1 Overall System Flow

```
Student types a message
       |
       v
+---------------------------+
|    Streamlit Chat UI      |
|  (chat bubbles, history,  |
|   starter prompts)        |
+---------------------------+
       |
       v
+---------------------------+
|   app.py  (handler)       |
|  trims message window     |
|  calls coach_app.invoke()  |
+---------------------------+
       |
       v
+----------------------------------------------+
|         LANGGRAPH AGENT (graph.py)           |
|                                              |
|   MASTER NODE  (plan-then-execute)           |
|       |                                      |
|  +---------+-----------+-----------+------+ |
|  |         |           |           |      | |
|  v         v           v           v      | |
| ANALYSER RETRIEVER  PLANNER    QUIZZER    | |
|  (ML)    (RAG)      (7-day      (MCQ       | |
|                     plan)       quiz)      | |
|                                            | |
|               END NODE                     | |
|   (greetings, guardrails, final reply)     | |
+----------------------------------------------+
       |
       v
+---------------------------+
|  PostgreSQL (Neon)        |
|  chat_sessions            |
|  chat_messages            |
|  chat_agent_state         |
+---------------------------+
```

2.2 Technology Stack

Component	Technology	Why
Agent framework	LangGraph	Required by spec. Lets us define a proper graph of nodes with state.
Language model	Groq API — Llama 3.3 70B	Free tier. Fast. Good at structured output and reasoning.
Embeddings	all-MiniLM-L6-v2 (Sentence Transformers)	Local, no API cost. Converts text to vectors for FAISS search.
Vector store	FAISS (faiss-cpu)	Fast semantic search over the local knowledge base.
Web search	Tavily API (optional)	Fallback when local knowledge base does not cover a topic.
ML models	Scikit-Learn .pkl files (M1)	All three Milestone 1 models reused directly.
UI	Streamlit	Extends the existing Milestone 1 dashboard.
Database	PostgreSQL via Neon	Persistent chat history across sessions. Survives redeployments.
Hosting	Streamlit Community Cloud	Public deployment. Free tier.

3. Agent State

All nodes in the graph share one state object. Think of it as a shared clipboard. When the Analyser node writes the ML result to state, the Planner node can read it later without needing the Analyser to pass it manually. This is how multi-node systems coordinate without tangled hand-offs.

The state is defined as a Python TypedDict. The messages field uses LangGraph's operator.add annotation, which means every node appends to the list rather than replacing it.

Field	Type	What it holds
messages	List[BaseMessage]	Full conversation so far (HumanMessage + AIMessage). Appended on every turn.
student_data	dict	Student profile built up from the conversation: {math: 68, reading: 72, test_prep: 'none', ...}
ml_results	dict	Latest model outputs: predicted_score, status, category, assumed_defaults.
retrieved_docs	List[str]	Documents returned by the last FAISS or web search. Used by Planner and Quizzer.
last_retrieval_query	str	The query used for the last retrieval. Avoids re-running the same search.
quiz_questions	List[dict]	The 5 MCQ questions generated for the current quiz session.
current_q_idx	int	Tracks which question is being answered (0-indexed).
quiz_score	int	Number of correct answers so far in the current quiz.
quiz_active	bool	True when a quiz is in progress. Locks the master node into quizzer routing.
awaiting_answer	bool	True after questions are shown. Tells the system the student needs to respond.
study_plan	str	The generated Markdown study plan. Stored for display.
plan	List[str]	The execution plan for this turn: e.g. ['analyser', 'planner', 'end'].
current_step_index	int	Which step of plan[] to execute next.
next_node	str	The node name the master node writes. LangGraph reads this to route
task_plan	List[dict]	Detailed reasoning for each step generated by LLM planner
response_parts	List[dict]	Stores outputs from each node for final composition

Field	Type	What it holds
response_mode	str	Classification: academic, greeting, off_topic, integrity_redirect
direct_response	str	Predefined response for greeting or guardrail cases

4. Orchestration — How the Master Node Works

The master node no longer uses deterministic keyword routing.
It fully relies on structured LLM planning using ExecutionPlan.

Each task includes:

- node name
- task description (why needed)

This enables:

- explicit reasoning trace (task_plan)
- explainable execution
- better debugging and evaluation

4.1 The Problem with Pure Dynamic Routing

An obvious approach is to have the master node call the LLM after every node execution to decide what comes next. This sounds flexible but causes a real problem: after the Analyser runs and appends its output to the messages list, the master node reads those messages again and the LLM sees the analysis output and thinks analysis is still needed. This creates an infinite loop where the same node keeps getting called.

This actually happened during development. The fix was the plan-then-execute pattern.

4.2 Plan-Then-Execute

Instead of deciding the next step after every node, the master node now does this:

- **Step 1** — When a new message arrives, create a complete plan for the whole turn. Example: ['analyser', 'planner', 'end'].
- **Step 2** — Store that plan in state along with a step counter starting at 0.
- **Step 3** — Each time the master node is called again during the same turn, just read the next step from the plan and advance the counter. No LLM call needed.
- **Step 4** — When the plan is done, route to end.

This means the LLM is only called once per user message for planning, not once per node transition. It is much faster, much cheaper, and completely eliminates the infinite loop.

4.3 How the Plan Gets Created

The master node now relies fully on an LLM-based structured planner.
Instead of keyword-based deterministic routing, the system uses a Pydantic schema (ExecutionPlan) to let the LLM reason over the **entire user intent**, including multi-intent queries.
The LLM generates:

- response_mode (academic / greeting / off_topic / integrity_redirect)
- ordered list of tasks (nodes)
- reasoning for why each step is required
- optional direct response

This ensures:

- correct handling of multi-intent queries
- better generalization to unseen inputs

4.4 Plan-Then-Execute Flow Diagram

```
 New user message arrives
        |
        v
 Is quiz_active = True?
   YES --> route directly to quizzer (bypass everything)
   NO  --> continue
        |
        v
 Does a valid plan[] exist with steps remaining?
   YES --> read plan[step_index], advance counter, route there
   NO  --> create a new plan
        |
        v
system calls the LLM planner for every new user message (except quiz-active cases).
        |
        v
 Apply guardrails to the plan
   - Remove invalid nodes
   - Remove duplicates
   - Skip analyser if ml_results exists and no new data
   - Skip retriever if same query was retrieved already
   - Append 'end' if missing
        |
        v
 Store plan in state, set step_index = 0
 Route to plan[0]
```

```
        |
        v
Specialist node runs, returns to master
Master reads next step from plan[]
Repeat until plan is complete
        |
        v
Route to END
```

4.5 Multi-Step Example

When a student types: "I got 55 in math and 62 in reading, can you make me a study plan?" — the plan created is:

```
Plan: ['analyser', 'retriever', 'planner', 'end']
```

The Analyser runs first and updates ml_results with the prediction and learner category. Then the Retriever searches the knowledge base for study strategies relevant to that category. Then the Planner reads both ml_results and retrieved_docs and builds a personalised 7-day plan. The End node sends the final response. All of this happens from one user message.

5. The Specialist Nodes

Each node is a Python function. It reads what it needs from state, does its job, and writes its output back to state. Nodes never call each other directly — all routing goes through the master node.

5.1 Analyser Node

The Analyser is the bridge between natural language and the Milestone 1 ML models. It does two things.

Data Extraction

It uses the LLM with a strict Pydantic schema (StudentDataSchema) to pull out any student profile data from the message. The schema covers math score, reading score, study hours, parent education level, test preparation status, lunch type, sport participation, gender, siblings, first-child status, and transport mode. The LLM is told to only extract values that are explicitly stated. If a value is not mentioned, it returns None for that field.

Extracted values are merged into the existing student_data in state. This means if the student said their math score in message 1 and their study hours in message 4, both values are available when the ML runs.

ML Pipeline

After extraction, all three Milestone 1 models run on the merged student data. Missing fields are filled with dataset medians (math = 67, reading = 70, study hours = 7.5, and so on). The result tells the student their predicted exam score, Pass or Fail status, and learner category. The response also tells the student which fields were assumed so nothing is hidden.

The cluster label mapping uses argsort on the KMeans cluster centres by ExamScore rather than hardcoding {0: At-Risk, 1: Average, 2: High-Performer}. This is important because KMeans cluster IDs are arbitrary integers that can change between training runs.

5.2 Retriever Node (RAG)

The Retriever is responsible for finding relevant study content before the system gives advice. It never invents information — everything it says is grounded in retrieved documents.

Local Knowledge Base

The knowledge base is a curated set of academic resources (math, reading, writing, study methods, exam strategies), including trusted sources like Khan Academy, ReadTheory, and Desmos.

All documents are chunked into paragraphs, embedded at application startup using the all-MiniLM-L6-v2 sentence transformer model, and loaded into a FAISS IndexFlatL2 structure. This runs once when the app starts. After that, every search is a fast in-memory vector comparison with no network call.

How Retrieval Works

The query is embedded and the top 4 documents are retrieved using L2 distance. Since semantic similarity alone can return unrelated matches, a lexical overlap check ensures the results actually share words with the query—otherwise they're discarded as irrelevant.

Web Search Fallback

If the local knowledge base lacks relevant content, the system falls back to Tavily web search (when enabled), targeting academic sources and clearly labeling the response. If neither source is useful, it responds honestly instead of fabricating information.

Retrieved documents are stored in retrieved_docs in state so the Planner can use them when building a study plan for the same turn.

5.3 Planner Node

The Planner generates a 7-day personalised study plan. It always requires retrieved documents to be present in state first. If retrieved_docs is empty, the Planner declines and explains why rather than making up a plan from scratch.

The plan is generated using a WeeklyPlan Pydantic schema with structured output, which guarantees a well-formed plan with exactly 7 days, each containing a topic, a list of specific

activities, and a time estimate. The LLM is told to use the retrieved documents as the content source for daily activities, not to invent syllabus content.

Pacing is adapted by learner category. At-Risk students get one concept per day, more revision time, and a full review day at the end. Average students get a moderate pace mixing theory and practice. High-Performers get multiple concepts per day and a challenge project on Day 7. If the learner category is Unknown, the system delivers a standard balanced plan without mentioning that data is missing.

The final plan is rendered as Markdown with the source documents credited, and stored in study_plan in state.

5.4 Quizzer Node

The Quizzer tests the student on a topic of their choice. It is the extension feature for Milestone 2.

How the quiz works

When the student asks for a quiz, the Quizzer first retrieves relevant documents from the knowledge base or web search for that topic. If no documents are found, it declines rather than generating ungrounded questions.

It then generates exactly 5 multiple-choice questions using a QuizSet Pydantic schema. Each question has exactly 4 options and one correct answer. The questions are based on the retrieved documents, so answers are traceable to the knowledge base.

All 5 questions are shown at once. The student responds with all answers in one message, for example: 1. A, 2. C, 3. B, 4. D, 5. A. The system parses this response and grades each answer by comparing the selected option text against the correct answer text. Feedback is given for every question with an explanation of why each answer is right or wrong.

At the end the student sees their total score, a closing message based on performance (perfect score, good effort, or keep practising), and a list of the concepts they should revise based on what they got wrong.

5.5 End Node

The End node is the last step in every execution plan. The End node composes the final response using `response_parts`, which contain outputs from all specialist nodes.

This design solves a key issue in multi-intent queries where earlier only the last node's output was visible.

Now:

- each node appends its output to `response_parts`
- the End node merges them into a single structured response

Example:

```
response_parts = [
  performance analysis,
  concept explanation,
  study plan
]
```

→ final composed answer includes all parts

5.6 Response Composition Layer

The system uses a response composition mechanism using `response_parts`.

Each specialist node does not directly generate the final user-visible answer. Instead, it contributes a structured output to `response_parts`.

The End node then:

- merges all parts
- ensures no information loss
- produces a unified final answer

This is critical for:

- multi-intent queries
- modular reasoning
- clean separation of concerns

6. Guardrails and Output Formatting

Two supporting modules keep the system focused and the output readable. These are not afterthoughts — they are part of the production architecture.

6.1 Academic Guardrails (guardrails.py)

Guardrails are implemented **inside the LLM prompts**, not as a separate keyword-based filtering system.

The master planner enforces:

- academic-only scope
- cheating detection (integrity_redirect)

- off-topic rejection
- greeting handling

These are returned as structured fields (response_mode, direct_response) from the LLM itself.

making guardrails:

- scalable
- context-aware
- robust to phrasing variations

Fixed responses for violations are still enforced at the end node to prevent prompt injection.

6.2 Output Formatting (formatting.py)

Language models sometimes return one long paragraph with no line breaks. This looks fine in a terminal but renders poorly in a chat UI. The polish_assistant_markdown function detects this and splits the response into readable blocks at sentence boundaries.

The rule is: if the response has no newlines and is longer than 320 characters, split it into groups of 3 sentences separated by blank lines. If the response already has newlines or is short, leave it alone. This is applied to every assistant message before it is displayed and before it is saved to the database.

7. Session Memory and Chat Persistence

The system maintains two kinds of memory. Within a conversation, the LangGraph state accumulates everything the student has said and everything the agent has produced. Across conversations, a PostgreSQL database stores the full message history so previous sessions can be resumed.

7.1 Within-Session Memory

Student profile data accumulates across the whole conversation. If a student mentions their math score in the first message and their reading score four messages later, both values are available when the ML runs because they are stored in student_data in state. The student never needs to repeat themselves.

To avoid sending the entire conversation history to the LLM on every turn (which would be slow and expensive), a trim_messages function caps the LangChain message window at 20 messages. The full conversation is still stored in the database, but the LangGraph graph only sees the recent 20. This keeps the context relevant without losing old session data.

7.2 PostgreSQL Database (Neon)

Chat history is stored in a PostgreSQL database hosted on Neon. This was chosen over SQLite because Streamlit Cloud's file system resets on every redeployment, which would wipe a SQLite file. PostgreSQL persists independently.

Three tables are used:

- **chat_sessions** — one row per conversation, storing the session ID, title (auto-generated from the first message), and timestamps.
- **chat_messages** — every message in every conversation, with role (user or assistant) and content.
- **chat_agent_state** — the serialised LangGraph state for each session, so when a student returns to an old conversation the agent knows their learner category, their student profile, and where the quiz left off.

The tables are created automatically on first startup using CREATE TABLE IF NOT EXISTS, so there is nothing to configure in the database manually. The connection reads the DATABASE_URL environment variable, which is set in Streamlit Cloud Secrets or a local .env file.

7.3 Chat UI

Messaging-style UI: sidebar chats, click to resume with full state; new chat, rename/delete options.

When the conversation is empty, example starter prompt chips are shown so the student knows what they can ask. Once messages exist, these disappear and the chat history takes over.

8. How the System Meets the Evaluation Criteria

Criterion	How we meet it
Quality of personalised study plans	Planner generates a 7-day plan via a Pydantic schema: each day includes a topic, actionable tasks from the knowledge base, and time estimates. It adapts to learner type (At-Risk: fundamentals, shorter sessions; High-Performer: advanced challenges) and cites source documents.

Criterion	How we meet it
Agent reasoning and state management	The system uses structured LLM planning (ExecutionPlan) to reason over full user intent, including multi-intent queries. The execution is state-driven and modular, with outputs accumulated in response_parts and composed at the end.
Relevance of retrieved resources (RAG)	FAISS retrieves top 4 documents, refined by a lexical filter to remove irrelevant matches. If needed, Tavily adds external sources. Retrieved docs are stored in state and reused by the Planner, enabling RAG across the system, not just the Retriever.
User experience and deployment reliability	Streamlit UI uses `st.chat_message`, a sidebar for history and starter prompts, and friendly error handling with logged IDs. The app is deployed at predictive-learning-analytics-ml.streamlit.app with PostgreSQL persistence across redeployments.

9. Key Design Decisions

Decision	What we did	Why
Plan-then-execute over dynamic routing	Master node creates the full plan once per turn, then executes steps deterministically.	Pure LLM-per-step routing caused infinite loops because completed work was re-evaluated as pending. Plan-then-execute separates planning from execution and eliminates loops entirely.
Full LLM-based Planning	structured LLM planning.	Improves handling of multi-intent queries, and uni intent queries
Prompt-based Guardrails	prompt-level enforcement inside the LLM planner and composer.	This makes safety checks context-aware and scalable.
Response Parts Architecture	response_parts to accumulate outputs from multiple nodes.	Ensures all user intents are fulfilled in a single response.
Lexical overlap filter in RAG	Semantic search results with no word overlap with the query are discarded.	FAISS returns the closest vector even when nothing is relevant. Without this filter, unrelated documents would be passed to the LLM and generate hallucinated-sounding but confident advice.
Planner refuses without retrieved docs	If retrieved_docs is empty, the Planner declines and explains why.	A plan built from nothing but LLM knowledge is not grounded in the actual study material. This forces the system to only plan from verified content.
Dynamic cluster label mapping	np.argsort on cluster centres by ExamScore at startup, not hardcoded {0, 1, 2}.	KMeans cluster IDs are arbitrary. Hardcoding them breaks silently when the model is retrained with a different random seed.

Decision	What we did	Why
PostgreSQL over SQLite	Neon PostgreSQL with DATABASE_URL environment variable.	Streamlit Cloud's filesystem resets on redeployment. SQLite files are wiped. PostgreSQL persists independently of the deployment.
End node does not re-synthesise AIMessage	If the last message in state is already an AIMessage, end_node returns empty.	Without this check, the End node would call the LLM to summarise a response the Retriever or Planner already wrote perfectly, often adding irrelevant sections.
Message window trimming	LangGraph state is capped at 20 messages using trim_messages.	Sending the full history on every LLM call grows token cost linearly with conversation length. Trimming keeps cost flat while the full history stays in the database.

10. Limitations and Future Work

10.1 Current Limitations

- **Knowledge base coverage** — the current documents cover core academic topics well but not every subject. Highly specific questions outside the knowledge base will fall back to web search or a gap message.
- **Web search is optional** — Tavily requires an API key. Without it, the system is entirely dependent on the local knowledge base.
- **No user accounts** — all conversations are stored in one shared database without authentication. Multiple users share the same session list. A login system would be needed for production.
- **Quiz is all-at-once** — all 5 questions are shown together and answered in one message. A one-question-at-a-time flow with adaptive branching would be better pedagogically.
- **Cold start time** — loading sentence transformers, FAISS, and six ML model files on first startup takes 10–20 seconds on Streamlit Cloud. Subsequent users on the same running container are fast.

11. Final Deliverables Summary

Deliverable	Requirement	Status
Publicly hosted application	Mandatory	https://predictive-learning-analytics-ml.streamlit.app/
LangGraph agent framework	Required	Multi-node graph with plan-then-execute master node
RAG implementation	Chroma or FAISS required	FAISS with all-MiniLM-L6-v2 + Tavily web search fallback

Deliverable	Requirement	Status
Session memory	Required	PostgreSQL (Neon) with three tables, full state persistence
Personalised study plans	Required	Planner node with structured 7-day output, RAG-grounded activities
Conversational interface	Required	Streamlit chat with history sidebar, starter prompts, session restore
Extension feature	Pick one	Knowledge Quiz — 5 MCQ questions grounded in knowledge base, auto-graded
GitHub repository	Required	https://github.com/sathvik89/Predictive-Learning-Analytics_ML
Demo video	5–7 minutes	[To be recorded and linked]
LLM	Free tier	Groq API — Llama 3.3 70B Versatile

Built for Gen AI Course — Milestone 2 \ Sathvik Koriginja, Anushka Tyagi, Apoorva Choudhary